

解答 1

1. ソフトウェア工学の概論

_____とは、人、機械、コンピュータが互いに協力し、ある_____を指す。

情報システム

目的を達成する仕組み全体

ソフトウェア

プログラム

データ

ドキュメント

_____は人間の_____システムであり、相手が非専門家であることが多いため、_____や_____が重要だ。一方、_____は_____システムであり、相手が専門家であることが多いため、_____や_____が重要となる。

情報システム

業務を支える

使いやすさ

わかりやすさ

組込みシステム

物理現象に働きかける

安全性

リアルタイム性

ソフトウェアは、対象によって作り変えられることがあり、その結果_____になる。また、要求の増加や機能追加により、_____や_____が進んでいる。さらに、ビジネスモデルの変化や技術の更新への適応が求められるため、_____。

多種多様

大規模化

複雑化

変化が激しい

_____とは、_____、_____、_____のことである。ソフトウェアの良し悪しはプロジェクトの成否に直結し、規模が大きくなるほど計画通りに進めるのが難しくなる。

QCD

品質

コスト

納期

_____は、コストなどの制約のもとでソフトウェアを開発・利用するための_____。その目的は、ソフトウェア開発に関わる問題を_____することだ。これは経験則に基づいている。コンピュータの性能向上により情報システムが構築され始め、その結果、開発の QCD などが問題になった。_____は「遅れている開発プロジェクトでは、開発要員の増員は遅れをひどくする」という法則だ。_____は_____と_____を掛け合わせたものだ。現代では、性能やメモリ利用効率を落としても、読みやすく修正しやすいコードが必要とされている。ソフトウェアモデルはプログラミング言語よりも抽象的に作成される。開発方法（開発の際の枠組み）や開発環境（IDE やコミュニケーションツール）も重要だ。コードやドキュメントの再利用やパターンを使うことで資産を生かすことができる（ライブラリなど）。

ソフトウェア工学

理論等を整理したもの

解決・改善

ブルックスの法則

工数

人数

期間

_____は、_____と_____からなる。要求定義では顧客から要求を聞き取り、要求事項としてまとめる。設計ではコードを記述するための方法を定義する。一方、_____は_____と_____で構成される。

上流工程

要求定義

設計

下流工程

実装

テスト

不具合の多くは_____で発生するが、その原因は_____にあることが多い。このため、上流工程からやり直す_____が発生し、工期に影響を与えることがある。### 管理や組織の問題_____は、時間、コスト、資源が限られた活動であり、_____は_____の管理を指す。規模が大きくなるにつれて、調整を行う管理業務の重要性が増し、コミュニケーション能力が重要なスキルとなる。### 変化の問題 ソフトウェアは再利用されることが多く、再利用のためにソースコードの追加や編集が繰り返される。その結果、コードが読みづらくなることもある。_____は業務の中核を支えるが、長期間の修正の繰り返しによって品質が劣化する。これをソフトウェアの「_____」と呼ぶ。

2. 情報システムとソフトウェア

_____は事実や概念、指示の_____であり、例えば文字コードがこれにあたる。一方、_____はデータを評価して得られる_____である。

データ

表現

情報

意味

ソフトウェアの動作は_____であり、CPUはクロック信号によって飛び飛びの値しか扱えない。また、OS がプログラムを切り替えながら動作する、一時停止を伴う動作を_____という。最終的には機械語で実行され、この一時停止動作も機械語単位で行われる。

クライアントサーバ方式

集中処理

分散処理

問題 1

1. ソフトウェア工学の概論					
_____とは、人、機械、コンピュータが互いに協力し、ある_____を指す。		_____とは、_____、_____、_____のことである。ソフトウェアの良し悪しはプロジェクトの成否に直結し、規模が大きくなるほど計画通りに進めるのが難しくなる。			不具合の多くは_____で発生するが、その原因は_____にあることが多い。このため、上流工程からやり直す_____が発生し、工期に影響を与えることがある。###管理や組織の問題_____は、時間、コスト、資源が限られた活動であり、_____は_____の管理を指す。規模が大きくなるにつれて、調整を行う管理業務の重要性が増し、コミュニケーション能力が重要なスキルとなる。###変化の問題 ソフトウェアは再利用されることが多く、再利用のためにソースコードの追加や編集が繰り返される。その結果、コードが読みづらくなることがある。_____は業務の中核を支えるが、長期間の修正の繰り返しによって品質が劣化する。これをソフトウェアの「_____」と呼ぶ。
_____は、_____、_____、そして関連する_____から構成される。		_____は、コストなどの制約のもとでソフトウェアを開発・利用するための_____。その目的は、ソフトウェア開発に関わる問題を_____することだ。これは経験則に基づいている。コンピュータの性能向上により情報システムが構築され始め、その結果、開発のQCDなどが問題になった。_____は「遅れている開発プロジェクトでは、開発要員の増員は遅れをひどくする」という法則だ。_____は_____と_____を掛け合わせたものだ。現代では、性能やメモリ利用効率を落としても、読みやすく修正しやすいコードが必要とされている。ソフトウェアモデルはプログラミング言語よりも抽象的に作成される。開発方法（開発の際の枠組み）や開発環境（IDE やコミュニケーションツール）も重要だ。コードやドキュメントの再利用やパターンを使うことで資産を生かすことができる（ライブラリなど）。			
_____は人間の_____システムであり、相手が非専門家であることが多いため、_____や_____が重要だ。一方、_____は_____システムであり、相手が専門家であることが多いため、_____や_____が重要となる。					
ソフトウェアは、対象によって作り変えられることがあり、その結果_____になる。また、要求の増加や機能追加により、_____や_____が進んでいる。さらに、ビジネスモデルの変化や技術の更新への適応が求められるため、_____。		_____は、_____と_____からなる。要求定義では顧客から要求を聞き取り、要求事項としてまとめる。設計ではコードを記述するための方法を定義する。一方、_____は_____と_____で構成される。			2. 情報システムとソフトウェア
					_____は事実や概念、指示の_____であり、例えば文字コードがこれにあたる。一方、_____はデータを評価して得られる_____である。
					ソフトウェアの動作は_____であり、CPU はクロック信号によって飛び飛びの値しか扱えない。また、OS がプログラムを切り替えながら動作する、一時停止を伴う動作を_____という。最終的には機械語で実行され、この一時停止動作も機械語単位で行われる。
					クライアントがサーバにアクセスする形式で、クライアントの数が多いシステム構成を_____と呼ぶ。一方、1 台のサーバで全ての処理を行う_____は効率的でデータの一貫性を保てるが、故障に弱く負荷が集中しやすい。これに対し、複数のサーバで処理を分担する_____は負荷軽減やバックアップが可能だが、データの一貫性やコストに課題がある。

1 台のコンピュータでシステムを構築する _____ は低コストだが、故障するとシステムが停止する。一方、 _____ は _____ と _____ で構成され、通常は別々の処理を行うが、故障時には従系が引き継ぐため、高コストながら故障に強い。また、2 台のコンピュータで同じ処理を同時に行う _____ は、高コストで効率は悪い。	シンプレックスシステム	3. 要求定義		目標を設定し、認識・整理する _____ では、直接目標を達成するのが難しいためサブゴールを設定する。サブゴールが全て満たされると上位も満たされると _____ と、サブゴールのうち 1 つでも満たされると上位も満たされる _____ がある。ゴールよりも具体的に一連の出来事を時系列で記述し、システム利用の流れや振る舞いの理解に役立つのが _____ である。その記述方法には、日常的な言葉で文章として書く _____、箇条書きや表で表現する _____、UML のシーケンス図やアクティビティ図に沿ってモデルを描く _____ がある。また、システムの利用のされ方を記述し、提供機能をグループ化できる _____ は、シナリオとゴールの中間の抽象度でシステムを大まかに把握するのに用いられる。これらの方法の一つに絞る必要はない。	ゴール分析
	デュプレックスシステム				
	主系				
	従系				
専用ハードウェアはハードウェアへの依存が強く、 _____ が低い。また、イベント発生後に応答する _____、時間制約を守る _____、 _____、継続して機能し続ける _____、人・モノ・環境に損害を与えない _____ といった特性が求められる。	デュアルシステム	_____ とは、顧客が欲しいものをもとに _____ をまとめる工程のことだ。そのプロセスは、 _____、 _____、 _____、そして _____ の順に進む。問題を解決したり目標を達成するために顧客が必要とする条件や能力を _____ と呼び、これは _____ と _____ に分類される。また、システムに影響を与えたり、システムから影響を受けたりする人、グループ、組織は _____ である。		要求定義	AND 分割
	移植性				
	リアクティブ性				
	リアルタイム性				
	資源の制約	顧客から情報を収集する _____ では、深く掘り下げられるが情報が未整理になりがちな _____、発展性に欠ける _____、複数のステークホルダが議論し気づきにくい要求も獲得できる _____ といった手法がある。また、 _____ や _____ を用いて発想を支援することもある。ステークホルダ間で利害が一致しない場合は、合意点を見つける必要がある。		要求仕様書	OR 分割
	信頼性				
	安全性				
		_____		要求獲得	シナリオ分析
		_____		要求分析	叙述的なシナリオ
		_____		要求仕様化	構造的なシナリオ
		_____		検証	ソフトウェアモデルの利用
		_____		要求	ユースケース分析
		_____		機能要求	
		_____		非機能要求	
		_____		ステークホルダ	
		_____		要求獲得	要求仕様化
		_____		インタビュー	正当性
		_____		アンケート	非あいまい性
		_____		ワークショップ	完全性
		_____		ブレーションストーミング	無矛盾性
		_____		プロトタイピング	順序付け
		_____			検証可能性
		_____			変更可能性
		_____			追跡可能性

問題 2

1 台のコンピュータでシステムを構築する _____ は低コストだが、故障するとシステムが停止する。一方、 _____ は _____ と _____ で構成され、通常は別々の処理を行うが、故障時には従系が引き継ぐため、高コストながら故障に強い。また、2 台のコンピュータで同じ処理を同時に行う _____ は、高コストで効率は悪い。																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																
---	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

4. 設計					
ソフトウェア開発における _____ とは、要求仕様書をもとに _____ を作成する工程である。この工程では、ソフトウェアの _____ や構成要素、それらの関係や特性を決定する。具体的には、外部設計とも呼ばれる _____ の決定、内部設計（アーキテクチャ設計）とも呼ばれ、構成要素とその関係を決める _____ の決定、そしてハードウェアや OS、使用技術などを定める _____ の決定が行われる。これらの決定後、設計の確認も重要なステップとなる。なお、プログラムの実現にはデータ構造とアルゴリズムが用いられる。		設計		モジュール内の結びつきの強さを示す _____ は、強度が強いほど良いとされる。その種類は強い順に、モジュールが 1 つの機能のみを提供する _____、特定データへのアクセスを行う機能をまとめた _____、手順的に共通データを参照・変更する _____、順番に実行される複数の機能を手順として 1 つのモジュールにする _____、ある時点で実行する機能がまとまっている _____、複数機能で 1 つが選択可能な _____、そして無秩序に機能を寄せ集めた _____ がある。	
ソフトウェア開発において、要求は「何をすべきか」という _____ を定義し、設計は「どのように実現するか」という _____ を定義する。要求は設計と対応している必要があり、これらは品質に影響を与える。プラットフォームは制約となり、要求に対して設計を割り当てると新たな問題が発生する可能性もある。		What How		モジュール間の結びつきの強さを示す _____ は、結合度が弱い（関係があまりない）ほど良いとされる。その種類は強い順に、他のモジュール内の構成要素に直接アクセスする _____、複数のモジュール間で読み書き可能な構造化されたデータ領域を参照する _____、複数のモジュール間で読み書き可能な変数を参照している場合の _____、他のモジュールの特定の制御を目的として if 文などの条件式で使用される引数を使う _____、共有変数で構造体データを受け取る場合の _____、共有変数でない変数を引数とする場合の _____ がある。モジュール間では情報隠蔽が重要であり、公開された操作のみを許可するこの概念を実現するのが _____ である。	
ソフトウェアの構成要素である _____ は、モジュールとその関係性を定義する _____ によって、その良し悪しが品質に直結する。モジュール化は作業分担の単位にもなり、再利用可能な構造にするためには _____ が良いとされる。また、適切な規模に分割することも重要だ。		モジュール モジュール化 モジュール性		モジュール結合度 内容結合 共通結合 外部結合 制御結合 スタンプ結合 データ結合 カプセル化	
_____ は、プログラムを _____、_____、_____ の 3 つの基本制御構造で記述する手法である。この手法では、実行速度が多少遅くなっても可読性を重視する。		構造化プログラミング 逐次 分岐 反復		モジュール強度 機能的 情動的 連絡的 手順的 時間的 論理的 暗号的	
				プロセスとしての _____ は、入力データを出力データに変換する。システム全体を一つの機能と捉え、それを細分化することを _____ という。機能分割は、最初は抽象的に行い、段階的に具体化していく。これを _____ と呼ぶ。データ同士の関係を定義する設計手法は、_____ である。イベントと現在の状態によって振る舞いや次の状態が決まるリアクティブシステムでは、_____ が用いられる。考慮漏れがないかを確認しやすいのは _____ だ。	
				独立性を高める目的で、クラス（設計図）やインスタンスをモジュール化の単位とする設計手法を _____ と呼ぶ。メソッド（振る舞い）とフィールド（データ）を定義するものを _____ という。フィールドへのアクセスを setter と getter のみに限定する考え方を _____ と呼ぶ。スーパークラスが持つフィールドやメソッドをサブクラスが引き継ぐ仕組みを _____ と呼ぶ。サブクラスでスーパークラスのメソッドを再定義できる（オーバーライド）性質を _____ という。_____ だが、渡されるデータの型が異なっても、型に応じた処理を実行できる機能は _____ と呼ばれる。	
				機能 機能分割 段階的詳細化 情報・データに注目する設計 状態に注目する設計 状態遷移表	
				オブジェクト指向設計 クラス カプセル化 継承 多態性 余談 オーバーロード	

問題 3

4. 設計					
ソフトウェア開発における____とは、要求仕様書をもとに____を作成する工程である。この工程では、ソフトウェアの____や構成要素、それらの関係や特性を決定する。具体的には、外部設計とも呼ばれる____の決定、内部設計（アーキテクチャ設計）とも呼ばれ、構成要素とその関係を決める____の決定、そしてハードウェアや OS、使用技術などを定める____の決定が行われる。これらの決定後、設計の確認も重要なステップとなる。なお、プログラムの実現にはデータ構造とアルゴリズムが用いられる。		モジュール内の結びつきの強さを示す____は、強度が強いほど良いとされる。その種類は強い順に、モジュールが 1 つの機能のみを提供する____、特定データへのアクセスを行う機能をまとめた____、手順的に共通データを参照・変更する____、順番に実行される複数の機能を手順として 1 つのモジュールにする____、ある時点で実行する機能がまとまっている____、複数機能で 1 つが選択可能な____、そして無秩序に機能を寄せ集めた____がある。		プロセスとしての____は、入力データを出力データに変換する。システム全体を一つの機能と捉え、それを細分化することを____という。機能分割は、最初は抽象的に行い、段階的に具体化していく。これを____と呼ぶ。データ同士の関係を定義する設計手法は、____である。イベントと現在の状態によって振る舞いや次の状態が決まるリアクティブシステムでは、____が用いられる。考慮漏れがないかを確認しやすいのは____だ。	
ソフトウェア開発において、要求は「何をすべきか」という____を定義し、設計は「どのように実現するか」という____を定義する。要求は設計と対応している必要があり、これらは品質に影響を与える。プラットフォームは制約となり、要求に対して設計を割り当てると新たな問題が発生する可能性もある。		モジュール間の結びつきの強さを示す____は、結合度が弱い（関係があまりない）ほど良いとされる。その種類は強い順に、他のモジュール内の構成要素に直接アクセスする____、複数のモジュール間で読み書き可能な構造化されたデータ領域を参照する____、複数のモジュール間で読み書き可能な変数を参照している場合の____、他のモジュールの特定の制御を目的として if 文などの条件式で使用される引数を使う____、共有変数で構造体データを受け取る場合の____、共有変数でない変数を引数とする場合の____がある。モジュール間では情報隠蔽が重要であり、公開された操作のみを許可するこの概念を実現するのが____である。		独立性を高める目的で、クラス（設計図）やインスタンスをモジュール化の単位とする設計手法を____と呼ぶ。メソッド（振る舞い）とフィールド（データ）を定義するものを____という。フィールドへのアクセスを setter と getter のみに限定する考え方を____と呼ぶ。スーパークラスが持つフィールドやメソッドをサブクラスが引き継ぐ仕組みを____と呼ぶ。サブクラスでスーパークラスのメソッドを再定義できる（オーバーライド）性質を____という。____だが、渡されるデータの型が異なっても、型に応じた処理を実行できる機能は____と呼ばれる。	
ソフトウェアの構成要素である____は、モジュールとその関係性を定義する____によって、その良し悪しが品質に直結する。モジュール化は作業分担の単位にもなり、再利用可能な構造にするためには____が良いとされる。また、適切な規模に分割することも重要だ。					
____は、プログラムを____、____、____の 3 つの基本制御構造で記述する手法である。この手法では、実行速度が多少遅くなっても可読性を重視する。					